UNIVERSITY OF EASTERN PANGASINAN
COLLEGE OF INFORMATION TECHNOLOGY
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

PowerShell & Ubuntu Command Manual

Copy-and-Paste Operational Handbook & Defense Execution Runbook
Enterprise Cloud Infrastructure

———————————————————————————

Student / Candidate: **Carlo Nimer**

Year & Section: Fourth Year – BSIT-4J

Course / Mission: **CCM101 (Cloud Computing) — Final Requirement (Mission 10)**

Instructor / Evaluator: Prof. Jenkielyn C. Torres

Academic Year: 2026–2027

Date of Submission: October 18, 2026

Handbook Overview & Operational Conventions

This handbook is designed as an immediate, copy-and-paste operational reference for configuring, inspecting, validating, and defending the Enterprise Cloud Infrastructure. Commands are explicitly separated by execution context:

[WINDOWS POWERSHELL]: Commands executed in Windows Terminal / PowerShell on the Lenovo LOQ host laptop.

[UBUNTU SSH]: Commands executed inside the remote Ubuntu Server 24.04 LTS guest OS over OpenSSH.

READ-ONLY: Commands that inspect, query, or display state without modifying any system configuration.

CHANGES CONFIGURATION: Commands that apply configurations, start/stop services, or create temporary test databases.

DESTRUCTIVE: Commands that drop tables or recreate volumes. Require explicit confirmation before execution.

Part A: Windows Host System & Hypervisor Verification

Execute these commands in Windows PowerShell on the Lenovo LOQ laptop before launching the virtual machine or starting the recorded technical defense.

A.1 Host Operating System & Architecture Audit

```
[WINDOWS POWERSHELL | READ-ONLY]
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion, OsArchitecture, CsName
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Queries Windows WMI/CIM tables for the registered OS edition, build release, system architecture, and hostname.
Operational Necessity:	Verifies that the physical host satisfies the baseline requirement of 64-bit Windows 11 with hypervisor support.
Expected Output:	WindowsProductName: Windows 11 Home / WindowsVersion: 2009 or 22H2/23H2 / OsArchitecture: 64-bit.
Output Interpretation:	Confirms 64-bit architecture capable of nested hardware-assisted virtualization (VT-x / AMD-V).
Troubleshooting / If Fails:	Ensure PowerShell 5.1+ is running as Administrator if WMI permissions are restricted.
State Impact:	Safe (Read-Only inspection)

A.2 Host Processor Model & Cores Audit

```
[WINDOWS POWERSHELL | READ-ONLY]
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores, NumberOfLogicalProcessors, MaxClockSpeed
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Retrieves physical CPU brand, hardware core count, and hyperthreaded logical processor count.

Operational Necessity:	Demonstrates physical CPU headroom before comparing it to the 2 vCPUs provisioned to the Ubuntu VM.
Expected Output:	Display of CPU (e.g., AMD Ryzen or Intel Core) showing 8+ cores and 12-16 logical processors.
Output Interpretation:	Proves that allocating 2 vCPUs to the guest leaves ample physical processing cores for the host OS.
Troubleshooting / If Fails:	Verify CimInstance permissions or fallback to 'wmic cpu get name,numberofcores'.
State Impact:	Safe (Read-Only inspection)

A.3 Physical RAM Conversion (Bytes to Gigabytes)

```
[WINDOWS POWERSHELL | READ-ONLY]
Get-CimInstance Win32_ComputerSystem | Select-Object
@{N="Physical_RAM_GB";E={[math]::Round($_.TotalPhysicalMemory / 1GB, 2)}},
@{N="Physical_RAM_Bytes";E={$_.TotalPhysicalMemory}}
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Calculates exact total physical memory in Gigabytes by dividing raw byte count by 1,073,741,824 (1 GB).
Operational Necessity:	Proves physical host RAM capacity (e.g., 16.0 GB or 32.0 GB) versus the 4.0 GB assigned to the VM.
Expected Output:	Physical_RAM_GB: 15.75 to 31.85 (accounting for hardware-reserved integrated GPU RAM).
Output Interpretation:	Shows that the 4 GB VM allocation occupies only ~25% of physical memory, ensuring zero host thrashing.
Troubleshooting / If Fails:	Run '(Get-CimInstance Win32_PhysicalMemory | Measure-Object Capacity -Sum).Sum / 1GB' as alternative.
State Impact:	Safe (Read-Only inspection)

A.4 OpenSSH Client Availability Verification

```
[WINDOWS POWERSHELL | READ-ONLY]
```

```
Get-Command ssh | Select-Object Name, Source, Version
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Locates the native Windows OpenSSH client executable path.
Operational Necessity:	Validates that SSH terminal administration can be conducted directly from PowerShell without third-party tools.
Expected Output:	Name: ssh.exe / Source: C:\Windows\System32\OpenSSH\ssh.exe.
Output Interpretation:	Native Windows OpenSSH client is present and registered in system PATH.
Troubleshooting / If Fails:	Install via Windows Settings > System > Optional Features > Add OpenSSH Client.
State Impact:	Safe (Read-Only inspection)

A.5 VirtualBox VM Registration & Inventory Audit

```
[WINDOWS POWERSHELL | READ-ONLY]
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" list vms
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Queries the VirtualBox hypervisor CLI for all registered virtual machines.
Operational Necessity:	Confirms the exact VM name identifier ('ubuntu-docker') required for scripted management.
Expected Output:	"ubuntu-docker" {xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}
Output Interpretation:	The 'ubuntu-docker' virtual machine is recognized and registered with the hypervisor.
Troubleshooting / If Fails:	Verify VirtualBox is installed in 'C:\Program Files\Oracle\VirtualBox'. If installed elsewhere, adjust path.
State Impact:	Safe (Read-Only inspection)

A.6 VirtualBox Virtual Hardware & Network Mode Inspection

```
[WINDOWS POWERSHELL | READ-ONLY]
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" showvminfo "ubuntu-docker" --machinereadable | Select-String -Pattern "memory=|cpus=|nic1=|bridgeadapter1="
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Extracts memory, CPU, and network adapter configuration flags directly from the VM definition file.
Operational Necessity:	Proves the VM is configured with 4096 MB RAM, 2 vCPUs, and Bridged networking to the physical Wi-Fi/NIC.
Expected Output:	memory=4096 / cpus=2 / nic1="bridged" / bridgeadapter1="Realtek Wi-Fi / Intel Wi-Fi Adapter"
Output Interpretation:	Confirms that the VM bridges into the physical LAN, allowing direct IP routability.
Troubleshooting / If Fails:	Ensure the VM name matches 'ubuntu-docker' exactly.
State Impact:	Safe (Read-Only inspection)

A.7 Host TCP Port 8088 Conflict Check

```
[WINDOWS POWERSHELL | READ-ONLY]
Get-NetTCPConnection -LocalPort 8088 -ErrorAction SilentlyContinue | Select-Object LocalAddress, LocalPort, State, OwningProcess
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Checks whether any local Windows service is already bound to port 8088.
Operational Necessity:	Ensures port 8088 is free for testing browser access to the Ubuntu VM.
Expected Output:	Empty output (indicating port 8088 is not occupied locally on Windows).
Output Interpretation:	No port conflict on the host machine; browser traffic to the VM will route cleanly.

Troubleshooting / If Fails:	If a process is listed, identify the process using 'Get-Process -Id <OwningProcess>' and terminate it.
State Impact:	Safe (Read-Only inspection)

Part B: Connecting PowerShell to Ubuntu Server via SSH

Follow this exact procedure to establish an authenticated terminal session from Windows PowerShell into Ubuntu Server.

B.1 Discovering the Active Ubuntu IP Address

```
[UBUNTU SSH (Console or Terminal) | READ-ONLY]
hostname -I | awk '{print $1}'
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH (Console or Terminal)
Action Performed:	Prints the primary IPv4 address leased by the local network DHCP server.
Operational Necessity:	Identifies the current destination IP for the SSH connection (e.g., 192.168.100.52 or previously observed 192.168.1.14).
Expected Output:	192.168.100.52 (or similar 192.168.x.x IPv4 address).
Output Interpretation:	This is the active IP assigned to interface enp0s3 / eth0. Use this address in PowerShell.
Troubleshooting / If Fails:	If no IP appears, run 'sudo dhclient -v' to request a lease from the router.
State Impact:	Safe (Read-Only inspection)

B.2 Initiating SSH Remote Session from PowerShell

```
[WINDOWS POWERSHELL | READ-ONLY]
ssh carlo@192.168.100.52
```

Analysis Field	Operational Guide
Where to Run:	WINDOWS POWERSHELL
Action Performed:	Initiates an encrypted OpenSSH session to user 'carlo' on the Ubuntu guest.
Operational Necessity:	Provides out-of-band remote administration directly from Windows.
Expected Output:	Password prompt: 'carlo@192.168.100.52's password:' followed by Ubuntu welcome banner.

Output Interpretation:	Session authenticated. The PowerShell prompt transforms into 'carlo@ubuntu-docker:~$'.
Troubleshooting / If Fails:	If connection timed out: verify VM is running and both devices share the same Wi-Fi subnet.
State Impact:	Safe (Read-Only inspection)

[NOTE] SSH Security & Terminal Behavior Notes:
1. First-Time Fingerprint: If prompted with 'The authenticity of host... ECDSA key fingerprint...', type 'yes' to store the host key in ~/.ssh/known_hosts.
2. Password Masking: When typing your Linux password, NO characters, asterisks, or dots will display on screen. This is standard UNIX security behavior to prevent shoulder-surfing. Type password steadily and press Enter.
3. Session Termination: To exit the SSH session and return to Windows PowerShell, type 'exit' and press Enter.

Part C: Ubuntu Guest Hardware & Resource Inspection

Execute these commands inside the Ubuntu SSH session to verify that virtual hardware matches the architectural design.

C.1 System Hostname, OS Release & Kernel Audit

```
[UBUNTU SSH | READ-ONLY]
hostnamectl
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Displays operating system, kernel version, architecture, and virtualization chassis details.
Operational Necessity:	Proves the guest is running Ubuntu Server 24.04 LTS (Noble Numbat) on x86_64 architecture.
Expected Output:	Static hostname: ubuntu-docker / Operating System: Ubuntu 24.04 LTS / Virtualization: oracle.
Output Interpretation:	Confirms that the guest recognizes it is running as an authorized VirtualBox guest.
Troubleshooting / If Fails:	Verify systemd is running normally.
State Impact:	Safe (Read-Only inspection)

C.2 Virtual CPU Core Allocation Inspection

```
[UBUNTU SSH | READ-ONLY]
nproc && lscpu | grep -E 'Model name|CPU\(s\):|Thread\(s\) per core|Core\(s\) per
socket'
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Counts available execution threads and queries the virtualized CPU topology.
Operational Necessity:	Verifies that exactly 2 virtual CPUs are allocated to the guest OS.

Expected Output:	2 / CPU(s): 2 / Thread(s) per core: 1 / Core(s) per socket: 2.
Output Interpretation:	The Linux kernel has access to exactly 2 execution threads, matching the VM allocation.
Troubleshooting / If Fails:	If only 1 CPU is visible, shut down the VM and increase processors in VirtualBox Settings > System > Processor.
State Impact:	Safe (Read-Only inspection)

C.3 Memory Allocation & Swap Space Audit

```
[UBUNTU SSH | READ-ONLY]
free -h && swapon --show
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Displays total, used, free, shared, buffer/cache, and available RAM, plus active swapfile partitions.
Operational Necessity:	Proves 4.0 GB provisioned RAM (~3.8 GiB usable) and active 2.8 GiB swap backing.
Expected Output:	Mem Total: ~3.8Gi, Used: ~1.2Gi, Available: ~2.4Gi. Swap: ~2.8Gi (file: /swapfile).
Output Interpretation:	Sufficient memory is available for all three container services with over 2 GiB headroom.
Troubleshooting / If Fails:	If swap is missing, enable it with 'sudo swapon -a' or inspect /etc/fstab.
State Impact:	Safe (Read-Only inspection)

C.4 Root Storage Disk Space Audit

```
[UBUNTU SSH | READ-ONLY]
df -h / && lsblk
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH

Action Performed:	Reports root filesystem capacity, used space, available space, and underlying block devices.
Operational Necessity:	Ensures adequate virtual disk space for container images, volumes, and gzip backup archives.
Expected Output:	/dev/mapper/ubuntu--vg-ubuntu--lv mounted on / with 14G+ size and >50% available free space.
Output Interpretation:	Sufficient disk capacity exists; backup pruning will prevent disk exhaustion.
Troubleshooting / If Fails:	If / is over 85% full, run 'docker system prune -a' to remove unused build layers.
State Impact:	Safe (Read-Only inspection)

Part D: Docker Multi-Tier Application Commands

Commands for managing, verifying, and observing the multi-tier container application.

D.1 Navigate to Project Directory & Validate Compose Spec

```
[UBUNTU SSH | READ-ONLY]
cd /home/carlo/enterprise-cloud && docker compose config -q && echo 'Compose
configuration valid.'
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Validates docker-compose.yml syntax, environment variable bindings, and network specifications.
Operational Necessity:	Pre-flight check preventing runtime failure due to YAML indentation or missing environment variables.
Expected Output:	Compose configuration valid. (exit code 0, no syntax errors).
Output Interpretation:	The infrastructure declaration is syntactically sound and ready for deployment.
Troubleshooting / If Fails:	Run 'docker compose config' without -q to view detailed parser errors and line numbers.
State Impact:	Safe (Read-Only inspection)

D.2 Inspect Container Health & Status

```
[UBUNTU SSH | READ-ONLY]
docker compose ps && docker ps --format 'table {{.Names}} {{.Status}}    {{.Ports}}'
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Lists running containers, their uptime, published ports, and automated health status.
Operational Necessity:	Proves that nginx, app, and db containers are running and healthy.
Expected Output:	nginx: Up (8088:80), app: Up (healthy, 5000), db: Up (healthy, 5432).

Output Interpretation:	All three microservices are operational; healthcheck probes are passing.
Troubleshooting / If Fails:	Check logs of failing container using 'docker compose logs <service>'.
State Impact:	Safe (Read-Only inspection)

D.3 Real-Time Container Resource Consumption Audit

```
[UBUNTU SSH | READ-ONLY]
docker stats --no-stream
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Takes a point-in-time snapshot of CPU percentage, memory usage, memory limits, and network I/O for all containers.
Operational Necessity:	Proves memory usage complies with configured resource limits (nginx: <96MB, app: <256MB, db: <384MB).
Expected Output:	Table showing low CPU (<2%) and low memory consumption across all 3 containers.
Output Interpretation:	Application microservices are lightweight and well within the 4 GB VM memory envelope.
Troubleshooting / If Fails:	If a container is near 100% memory limit, inspect application memory leak in app logs.
State Impact:	Safe (Read-Only inspection)

D.4 Live Application Telemetry & Health Probe Test

```
[UBUNTU SSH | READ-ONLY]
curl -i -sS http://127.0.0.1:8088/health
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Sends an HTTP GET request to the Nginx reverse proxy, which proxies to Flask,

	querying PostgreSQL socket health.
Operational Necessity:	Demonstrates end-to-end full-stack health: Nginx -> Flask -> PostgreSQL.
Expected Output:	HTTP/1.1 200 OK Content-Type: application/json {"database":"connected","status":"healthy"}
Output Interpretation:	Full-stack connectivity is operational. Sub-15ms database socket response confirmed.
Troubleshooting / If Fails:	If 502 Bad Gateway: check Flask app status. If database disconnected: check PostgreSQL container health.
State Impact:	Safe (Read-Only inspection)

Part E: Backup and Disaster Recovery Commands

Commands for verifying the automated database backup pipeline and executing non-destructive restore tests.

E.1 Audit Backup Directory & Compressed SQL Archives

```
[UBUNTU SSH | READ-ONLY]
ls -lah /home/carlo/enterprise-cloud/backups/
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Lists all generated backup archives with timestamps, file sizes, and Unix permissions.
Operational Necessity:	Confirms that backups are being created and permissions restrict access strictly to user carlo (077 umask).
Expected Output:	cloudapp_YYYYMMDD_HHMMSS.sql.gz files with -rw------- (0600) permissions and size > 2 KB.
Output Interpretation:	Backups are successfully written and secured against unauthorized local read access.
Troubleshooting / If Fails:	If directory is empty, run 'bash /home/carlo/enterprise-cloud/automation-script.sh' manually.
State Impact:	Safe (Read-Only inspection)

E.2 Verify Backup File Gzip Integrity

```
[UBUNTU SSH | READ-ONLY]
LATEST=$(ls -t /home/carlo/enterprise-cloud/backups/cloudapp_*.sql.gz | head -n 1) &&
gzip -t "$LATEST" && echo "$LATEST: INTEGRITY VERIFIED (OK)"
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Performs a complete Cyclic Redundancy Check (CRC-32) on the most recent compressed SQL backup.
Operational Necessity:	Proves the archive is uncorrupted and decompression will succeed during an actual disaster recovery event.

Expected Output:	...cloudapp_YYYYMMDD_HHMMSS.sql.gz: INTEGRITY VERIFIED (OK). Exit code 0.
Output Interpretation:	The gzip archive has zero bit rot or truncated bytes; database dump is fully intact.
Troubleshooting / If Fails:	If gzip returns an error, the file was truncated during backup. Check disk space.
State Impact:	Safe (Read-Only inspection)

E.3 Non-Destructive Disaster Recovery Restoration Test

```
[UBUNTU SSH | READ-ONLY]
LATEST=$(ls -t /home/carlo/enterprise-cloud/backups/cloudapp_*.sql.gz | head -n 1)
docker compose exec -T db psql -U postgres -c "DROP DATABASE IF EXISTS
cloudapp_restore_test;"
docker compose exec -T db psql -U postgres -c "CREATE DATABASE cloudapp_restore_test;"
gunzip -c "$LATEST" | docker compose exec -T db psql -U postgres -d
cloudapp_restore_test
docker compose exec -T db psql -U postgres -d cloudapp_restore_test -c "SELECT count(*)
AS total_restored_records FROM workloads;"
docker compose exec -T db psql -U postgres -c "DROP DATABASE cloudapp_restore_test;"
echo "Restoration verification passed."
```

Analysis Field	Operational Guide
Where to Run:	UBUNTU SSH
Action Performed:	Creates a temporary test database, streams the restored SQL dump into it, counts records, and cleans up.
Operational Necessity:	Verifies that the backup is functionally restorable without overwriting or disrupting the production database.
Expected Output:	CREATE DATABASE / table schema restored / total_restored_records count displayed / Restoration verification passed.
Output Interpretation:	Proves recovery capability satisfies Recovery Time Objective (RTO) under 60 seconds with 100% data fidelity.
Troubleshooting / If Fails:	Inspect PostgreSQL logs to determine if SQL syntax errors or role permissions failed.
State Impact:	Safe (Read-Only inspection)

Part F: Complete Defense-Day Step-by-Step Command Sequence

Execute this exact 18-step sequence during your recorded technical defense. Follow the sequence sequentially from Step 1 to Step 18:

Step #	Action Objective	Execution Terminal	Exact Command
Step 1	Windows Host Audit	WINDOWS POWERSHELL	Get-ComputerInfo | Select-Object WindowsProductName, OsArchitecture
Step 2	Host Memory Audit	WINDOWS POWERSHELL	Get-CimInstance Win32_ComputerSystem | Select-Object @{N="RAM_GB";E={[math]::Round($_.TotalPhysicalMemory/1GB,2)}}
Step 3	Hypervisor Check	WINDOWS POWERSHELL	& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" --version
Step 4	VM Definition Audit	WINDOWS POWERSHELL	& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" showvminfo "ubuntu-docker" --machinereadable | Select-String "memory=|cpus=|nic1="
Step 5	Establish SSH Session	WINDOWS POWERSHELL	ssh carlo@192.168.100.52 (substituting current active VM IP)
Step 6	Guest OS Verification	UBUNTU SSH	hostnamectl
Step 7	Guest Compute Audit	UBUNTU SSH	nproc && free -h && swapon --show
Step 8	Guest Storage Audit	UBUNTU SSH	df -h /
Step 9	UFW Firewall Status	UBUNTU SSH	sudo ufw status verbose
Step 10	Navigate to Project	UBUNTU SSH	cd /home/carlo/enterprise-cloud
Step 11	Compose Spec Validate	UBUNTU SSH	docker compose config -q && echo 'Config OK'
Step 12	Container Service Status	UBUNTU SSH	docker compose ps
Step 13	Resource Utilization	UBUNTU SSH	docker stats --no-stream

Step 14	Web Ingress HTTP Check	UBUNTU SSH	curl -i http://127.0.0.1:8088/
Step 15	Active Telemetry Probe	UBUNTU SSH	curl -sS http://127.0.0.1:8088/health
Step 16	Container Log Inspection	UBUNTU SSH	docker compose logs --tail=20 app
Step 17	Audit Backups & Cron	UBUNTU SSH	ls -lah backups/ && crontab -l
Step 18	Live Non-Destructive Restore	UBUNTU SSH	LATEST=$(ls -t backups/*.sql.gz | head -1) && gzip -t $LATEST && echo 'Archive verified.'